

PYTHON FÜR PROGRAMMIER-ANFÄNGER

Michael Aydinbas | Aydinbas IT Consulting & Training | info@aydinbas.training



MICHAEL AYDINBAS

Lead Data Engineer

info@aydinbas.training

AYDINBAS IT CONSULTING & TRAINING

DRIVE POSITIVE IMPACT

- MSc Psychology in IT, TU Darmstadt
- 4 years as data consultant at Exxeta
- 2 years as Senior Data Engineer at New Work SE
- Lead Data Engineer at Xibix Solutions GmbH
- Trainer for Python, Data Science, Machine Learning, Deep Learning and AI, Data Engineering
- Building AI services and scalable data platforms

WILLKOMMEN

PYTHON FÜR PROGRAMMIER-ANFÄNGER

Fünf Vormittage, Montag bis Freitag, 09:00–12:30 Uhr

Von der ersten Zeile Code bis zum eigenen Spiel: Variablen, Bedingungen, Schleifen, Collections, Funktionen, Dateien und Klassen

Begleitbuch: Michael Inden, Einstieg in Python, 2. Auflage, dpunkt 2024

Willkommen und Ziele	01	Werkzeuge & Setup	02
Notebooks und Übungen	03	Täglich Feedback	04
Abschlussprojekt Tic-Tac-Toe	05	Wie geht es weiter?	06


01 WILLKOMMEN

WILLKOMMEN

KENNENLERN-RUNDE

- Wie heißt du?
- Wo arbeitest du? Was ist dein Hintergrund?
- Hast du schon programmiert? Womit?
- Was möchtest du mit Python machen?
- Wann wäre die Woche für dich ein Erfolg?



Beginner:in — Anwender:in — Fortgeschritten — Expert:in

Ich habe noch keine Erfahrung mit der Thematik und habe auch noch nie damit gearbeitet

Ich kann jedes Problem innerhalb der Thematik eigenständig lösen und anderen erklären

ABLAUF UND SPIELREGELN

Zeiten

Montag bis Freitag, 09:00–12:30 Uhr, Pause gegen 10:30 Uhr und jederzeit nach Bedarf

Material

Notebooks mit leeren Übungen in notebooks/, Lösungen in loesungen/

Begleitbuch: Michael Inden, Einstieg in Python, 2. Auflage, dpunkt 2024

Arbeitsweise

Kurzer Input, dann selbst ausprobieren: jedes Notebook endet mit Übungen

Fragen jederzeit, Fehlermeldungen sind willkommen: wir lesen sie gemeinsam

Am Ende jedes Tages: kurzes Feedback

AGENDA

Wir gehen die Notebooks der Reihe nach durch; das Tempo bestimmt die Gruppe.

Thema	Darin
Erste Schritte	Notebooks, Hilfe, Stil
Variablen und Datentypen	Zahlen, Text, f-Strings
Bedingungen	Vergleiche, if, match
Schleifen	for, range, while
Strings	Slicing, Methoden
Collections	Listen, Dicts, Sets

Thema	Darin
Funktionen	Parameter, Rückgabe
Module und Pakete	import, eigene Module
Fehlerbehandlung	Tracebacks, try/except
Dateien, Datum und Zeit	Text, CSV, JSON, datetime
Klassen und Objekte	Klassen, Vererbung
Testen und Abschluss	pytest, Tic-Tac-Toe


02 WERKZEUGE &
SETUP

UNSERE WERKZEUGE

Auf den Schulungsrechnern hat die IT alles vorinstalliert. Wer mit dem eigenen Rechner arbeitet, findet auf den nächsten Folien Installation und Schnelleinstieg.

Werkzeug	Wofür	Läuft es?
Python 3.15	die Programmiersprache	`uv run python --version`
uv	Python, Pakete und Projekte verwalten	`uv --version`
VS Code	Editor für Skripte und Notebooks	`code --version`
Python- und Jupyter-Erweiterung	Notebooks in VS Code	`Strg+Shift+X`
Terminal	Befehle eingeben, Skripte starten	`pwd`

UV

uv installieren

```
# macOS / Linux
$ curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
# oder
> winget install --id=astral-sh.uv -e

# Test: neues Terminal öffnen
$ uv --version
```

Ein Werkzeug für Python-Versionen, Pakete und Projektumgebungen: ersetzt pip, venv und pyenv.

Installation

Befehl links im Terminal ausführen, danach das Terminal neu öffnen.

Anleitung

https://docs.astral.sh/uv/getting-started/installation/

Python selbst?

Installiert uv beim ersten uv sync automatisch, passend zum Projekt.

UV: DIE WICHTIGSTEN BEFEHLE

Befehl	Was passiert
`uv sync`	richtet die Projektumgebung .venv ein (einmal pro Projekt)
`uv run python skript.py`	startet ein Skript in der Projektumgebung
`uv run python`	öffnet die interaktive Python-Konsole
`uv add requests`	fügt ein Paket hinzu und trägt es in pyproject.toml ein
`uv remove requests`	entfernt ein Paket wieder
`uv python install 3.15`	installiert eine bestimmte Python-Version
`uv init mein-projekt`	legt ein neues Projekt an
`uv --version`	zeigt die installierte Version

WERKZEUGE & SETUP

VS CODE

VS Code einrichten

```
# Erweiterungen per Klick: Strg+Shift+X, dann suchen
#   Python   (Microsoft)
#   Jupyter  (Microsoft)
#   Ruff     (Astral)

# oder im Terminal:
$ code --install-extension ms-python.python
$ code --install-extension ms-toolsai.jupyter
$ code --install-extension charliermarsh.ruff

# Test
$ code --version
```

Kostenloser Editor von Microsoft: darin schreiben wir Skripte und arbeiten mit Notebooks.

Installation

https://code.visualstudio.com/

Installer ausführen; unter Windows den Haken bei „Zu PATH hinzufügen“ setzen.

Erweiterungen

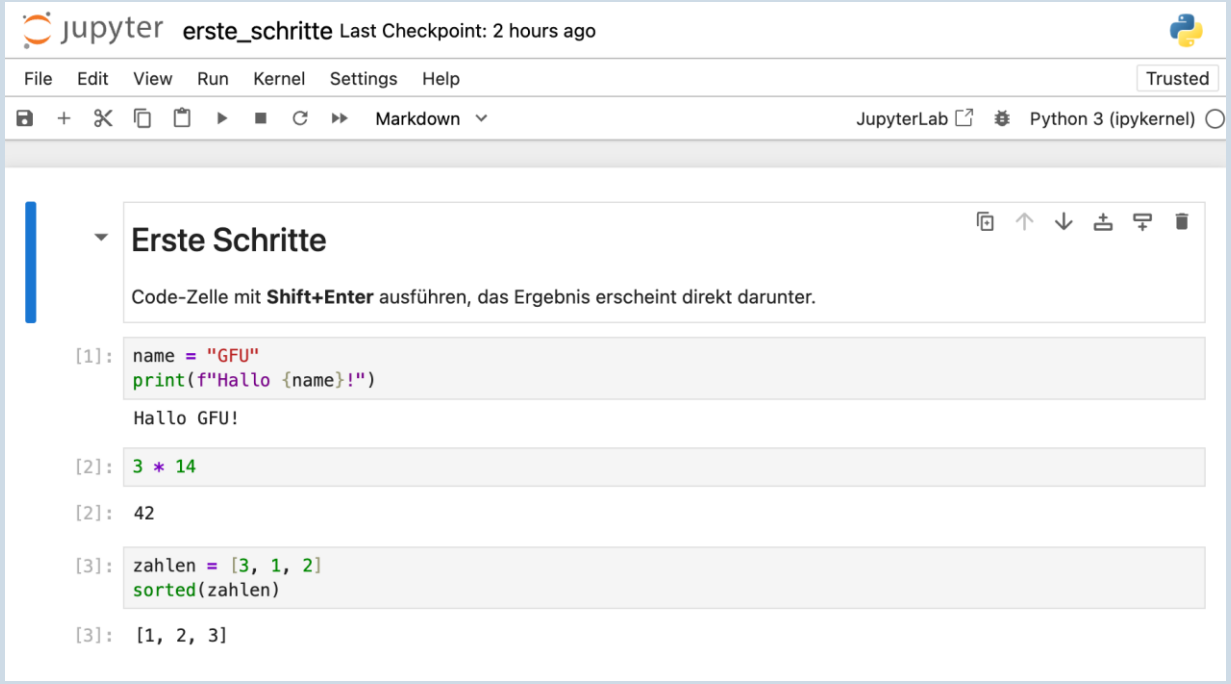
Python und Jupyter (Microsoft), Ruff (Astral)

macOS

Befehl code im Terminal: Befehlspalette, „Shell Command: Install ‚code‘ command in PATH“

VS CODE: DIE WICHTIGSTEN TASTENKÜRZEL

Windows / Linux	macOS	Aktion
`Strg+Shift+P`	`Cmd+Shift+P`	Befehlspalette: jeden Befehl suchen
`Strg+P`	`Cmd+P`	Datei nach Namen öffnen
`Strg+Shift+X`	`Cmd+Shift+X`	Erweiterungen
`Strg+S`	`Cmd+S`	Speichern
`Strg+F`	`Cmd+F`	Suchen in der Datei
`Shift+Alt+F`	`Shift+Option+F`	Code formatieren
`F5`	`F5`	Skript mit Debugger starten
`Menü Terminal`	`Menü Terminal`	Neues Terminal im Projektordner



WERKZEUGE & SETUP

JUPYTER NOTEBOOKS

Start 1: in VS Code

Notebook öffnen, „Select Kernel“, dann .venv

Start 2: im Browser (optional)

Einmalig im Kursordner:

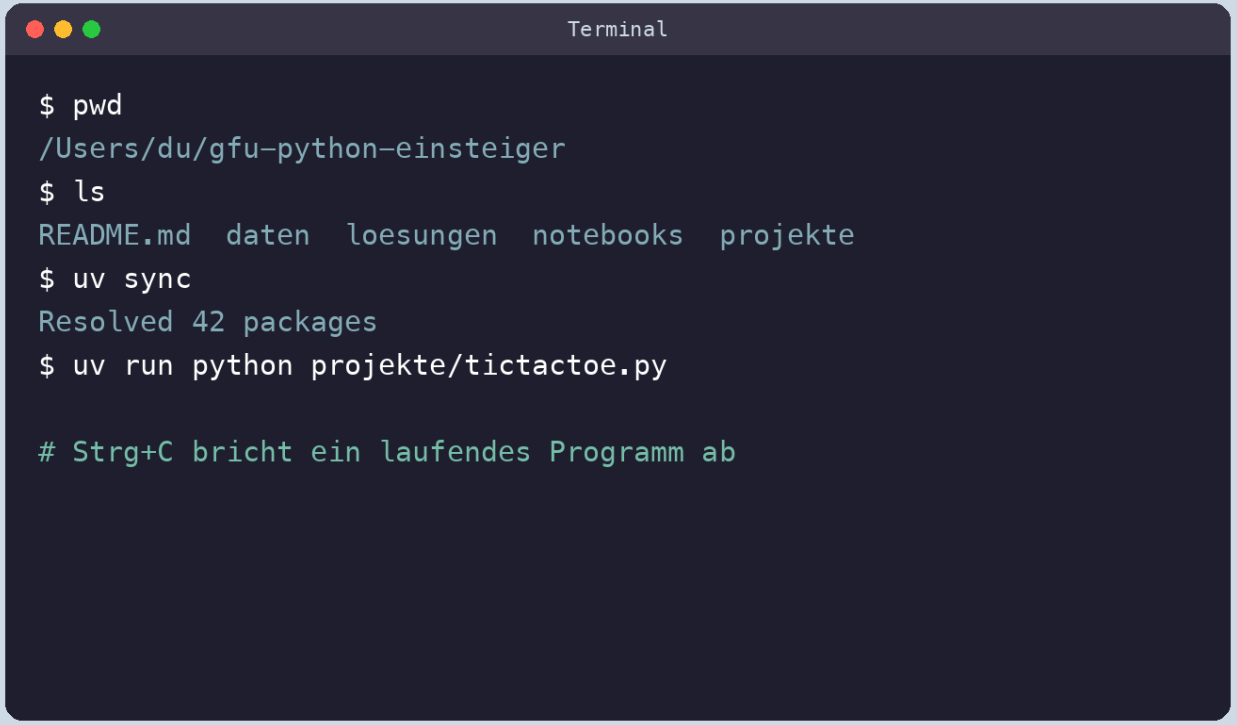
```
uv run python -m ipykernel install --user --name kurs
--display-name Kurs
```

Starten, dann Kernel „Kurs“ wählen:

```
uvx --python 3.14 --from notebook jupyter-notebook
```

NOTEBOOKS: DIE WICHTIGSTEN TASTEN

Taste / Button	Aktion
`Shift+Enter`	Zelle ausführen und zur nächsten springen
`Strg+Enter`	Zelle ausführen und stehen bleiben
`Esc / Enter`	Befehlsmodus / Zelle bearbeiten
`A / B`	neue Zelle oberhalb / unterhalb (Befehlsmodus)
`M / Y`	Zelle zu Text (Markdown) / zu Code machen
`D D / X`	Zelle löschen
`Z`	Löschen rückgängig machen
`Button Restart`	Kernel neu starten (Symbolleiste): alle Variablen sind weg



WERKZEUGE & SETUP

TERMINAL

Textfenster für Befehle: hier laufen uv und unsere Skripte.

Windows

PowerShell oder die App Terminal (Startmenü)

macOS

App Terminal (Spotlight: Cmd+Leertaste, „Terminal“)

In VS Code

Menü Terminal, Neues Terminal: öffnet direkt im Projektordner

TERMINAL: DIE WICHTIGSTEN BEFEHLE

macOS / Linux	Windows PowerShell	Was passiert
`pwd`	`pwd`	Wo bin ich gerade?
`ls`	`ls oder dir`	Inhalt des Ordners anzeigen
`cd notebooks`	`cd notebooks`	in einen Ordner wechseln
`cd ..`	`cd ..`	eine Ebene nach oben
`mkdir test`	`mkdir test`	Ordner anlegen
`clear`	`cls`	Bildschirm leeren
`Tab / Pfeil hoch`	`Tab / Pfeil hoch`	Namen ergänzen / letzter Befehl
`Strg+C`	`Strg+C`	laufendes Programm abbrechen

KI ALS LERNHILFE

Was Studien zeigen

- KI, die Lösungen liefert: beim Üben besser, danach ohne KI 17 % schlechter (Bastani et al. 2025)
- Gut schneidet ab, wer sich Code erklären lässt und Verständnisfragen stellt; schlecht, wer alles abgibt oder die KI Fehler beheben lässt (Shen & Tamkin 2026)

So hilft sie

- Erst selbst versuchen, dann fragen
- Nach dem Warum fragen, nicht nach der Lösung
- Fehlermeldung zuerst selbst lesen, die KI höchstens erklären lassen
- KI-Code vor dem Ausführen vorhersagen, dann ändern
- Verständnis prüfen: ähnliche Aufgabe ohne KI lösen

So schadet sie

- Übung hineinkopieren und die Lösung übernehmen

TUTOR-PROMPT ZUM KOPIEREN

Als erste Nachricht in ChatGPT oder die KI deines Unternehmens kopieren. Code und Fehlermeldungen gibst du per Kopieren ins Chatfenster.

```
Du bist mein Python-Tutor. Ich lerne gerade programmieren
(Anfänger, Python 3.15, nur die Standardbibliothek). Regeln:
- Gib mir keinen fertigen Code für meine Aufgabe, auch nicht,
  wenn ich darum bitte. Kurze Beispiele zu einem Konzept
  (höchstens 3 Zeilen, nicht meine Aufgabe) sind erlaubt.
- Bring mich mit Rückfragen selbst auf die Lösung,
  immer nur ein Hinweis auf einmal.
- Bei Fehlermeldungen: erkläre, was sie bedeuten und wo ich
  suchen soll, aber nicht die Korrektur.
- Bei meinem Code: frag mich zuerst, was er ausgibt; sag mir
  dann, ob er stimmt, und wenn nicht, wo ich hinschauen soll.
- Erkläre einfach und auf Deutsch.
- Bitte mich am Ende, das Gelernte in einem Satz
  zusammenzufassen.
```

KURSMATERIAL EINRICHTEN

1. Kursmaterial herunterladen und entpacken (Link kommt im Kurs)
2. Im Ordner ein Terminal öffnen und `uv sync` ausführen
3. VS Code: Datei, Ordner öffnen, den Kursordner wählen
4. Ein Notebook aus `notebooks/` öffnen, Kernel `.venv` wählen
5. Erste Zelle mit Shift+Enter ausführen: erscheint eine Ausgabe, läuft alles

Gegenprobe im Terminal

```
uv run python projekte/tictactoe.py
```


03 MONTAG

MONTAG

LOS GEHT'S MIT PYTHON

Setup erledigt?

Notebook geöffnet, Kernel .venv gewählt, erste Zelle zeigt eine Ausgabe. Wenn nicht: jetzt melden, wir lösen es gemeinsam.

Ab jetzt arbeiten wir in den Notebooks

Start mit `notebooks/00_einfuehrung.ipynb`, danach der Reihe nach wie in der Agenda.

Eigene Notizen

Direkt ins Notebook schreiben: neue Zelle, mit M zu Text machen.

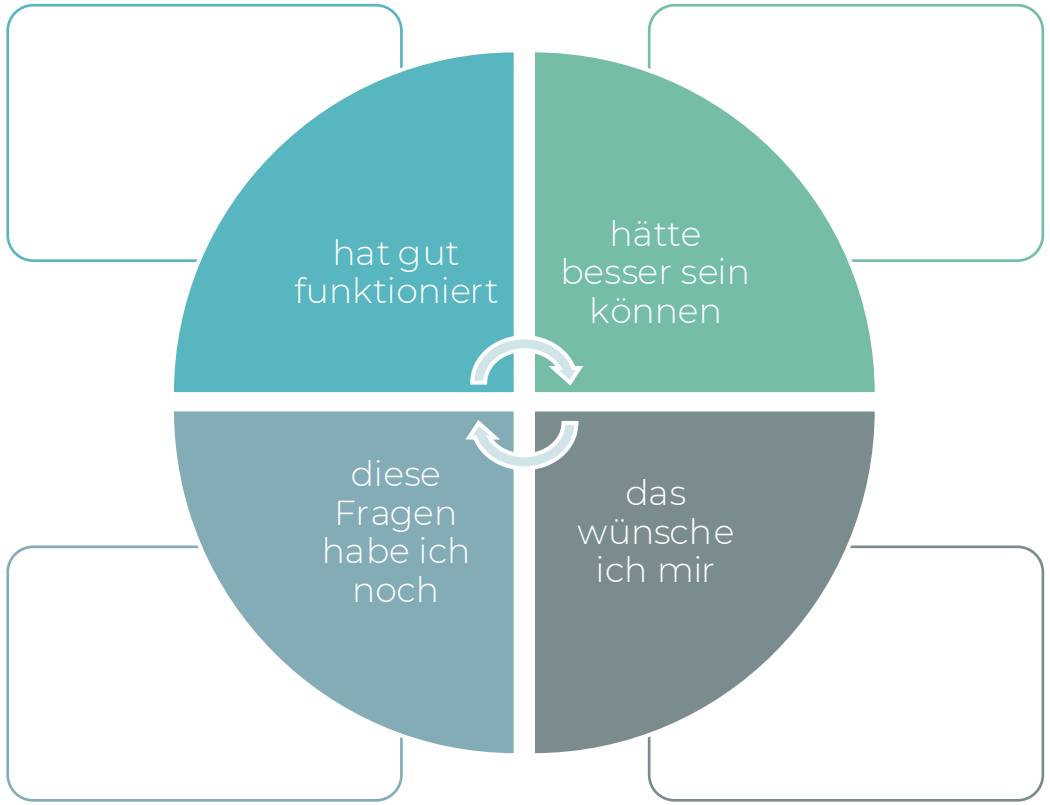
FEEDBACK TAG 1

WIE GEHT ES DIR?

Hier hast du die Möglichkeit, das heutige Seminar zu bewerten und mir so zu helfen, den nächsten Tag oder das nächste Seminar besser zu machen

zu langsam — Tempo — zu schnell

zu viel Theorie
zu wenig Praxis — Theorie vs Praxis — zu viel Praxis
zu wenig Theorie




04 DIENSTAG

RECAP TAG 1

RÜCKBLICK AUF MONTAG

Das haben wir gemacht

...

Das solltest du jetzt können

...

Code zum Merken

...

Frage zum Aufwärmen

...

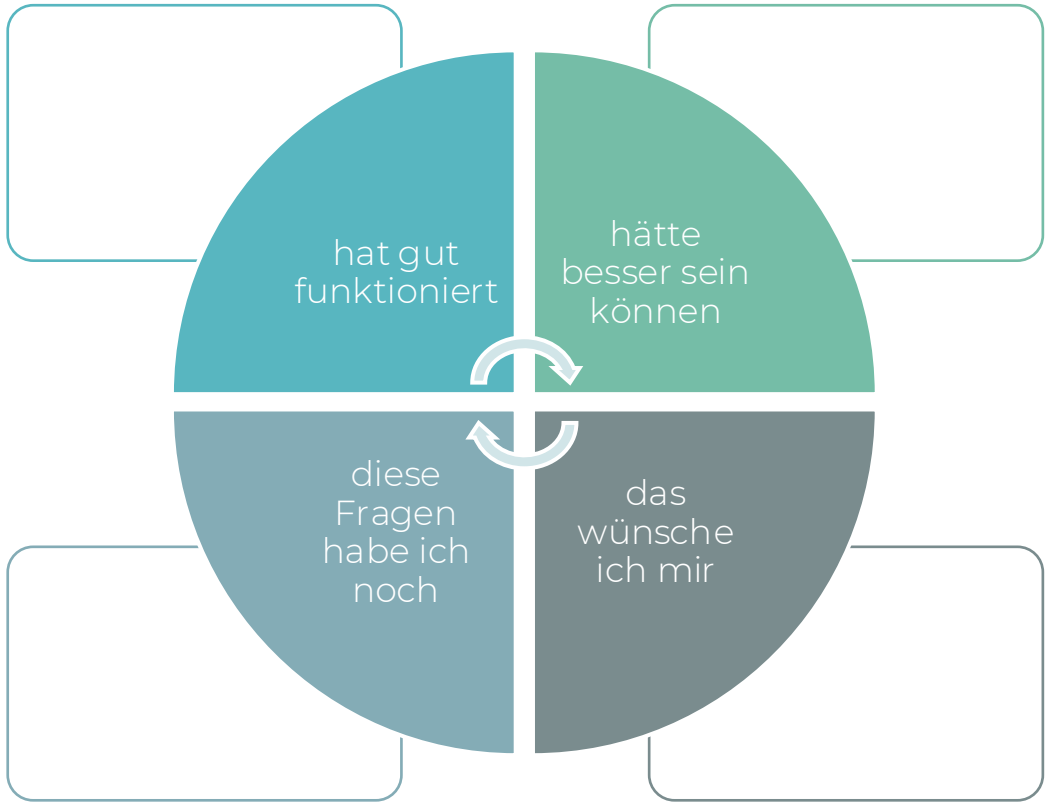
FEEDBACK TAG 2

WIE GEHT ES DIR?

Hier hast du die Möglichkeit, das heutige Seminar zu bewerten und mir so zu helfen, den nächsten Tag oder das nächste Seminar besser zu machen

zu langsam — Tempo — zu schnell

zu viel Theorie
zu wenig Praxis — Theorie vs Praxis — zu viel Praxis
zu wenig Theorie




05 MITTWOCH

RECAP TAG 2

RÜCKBLICK AUF DIENSTAG

Das haben wir gemacht

...

Das solltest du jetzt können

...

Code zum Merken

...

Frage zum Aufwärmen

...

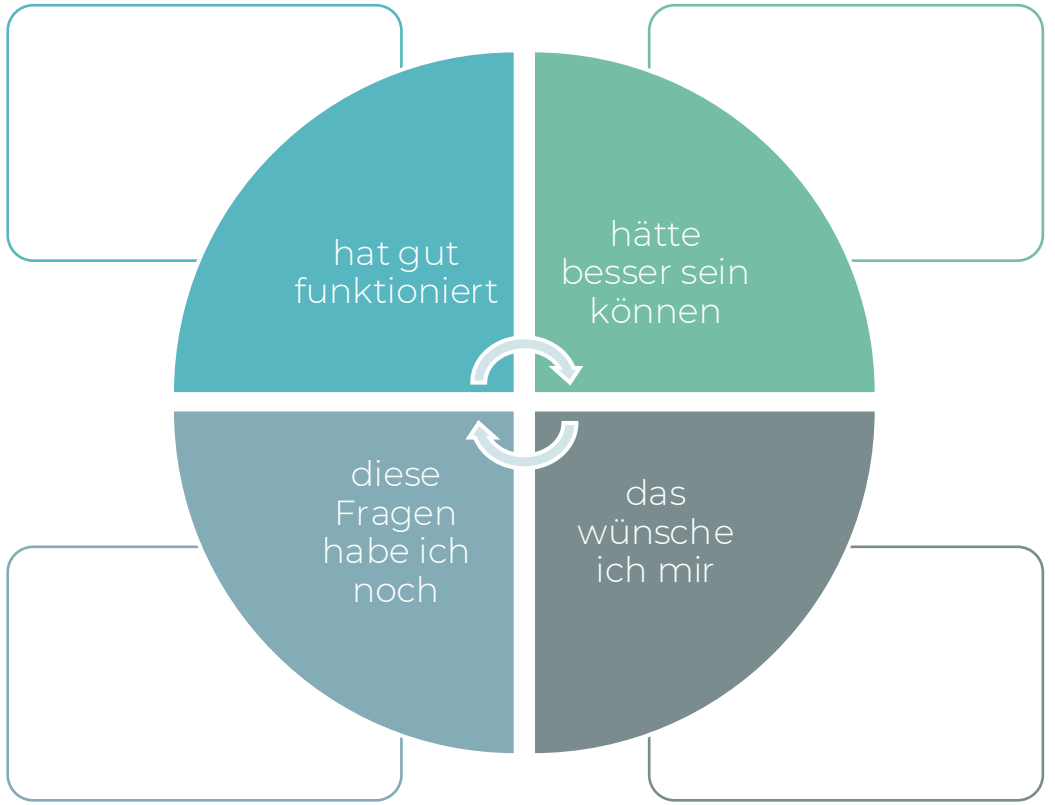
FEEDBACK TAG 3

WIE GEHT ES DIR?

Hier hast du die Möglichkeit, das heutige Seminar zu bewerten und mir so zu helfen, den nächsten Tag oder das nächste Seminar besser zu machen

zu langsam — Tempo — zu schnell

zu viel Theorie
zu wenig Praxis — Theorie vs Praxis — zu viel Praxis
zu wenig Theorie




06 DONNERSTAG

RECAP TAG 3

RÜCKBLICK AUF MITTWOCH

Das haben wir gemacht

...

Das solltest du jetzt können

...

Code zum Merken

...

Frage zum Aufwärmen

...

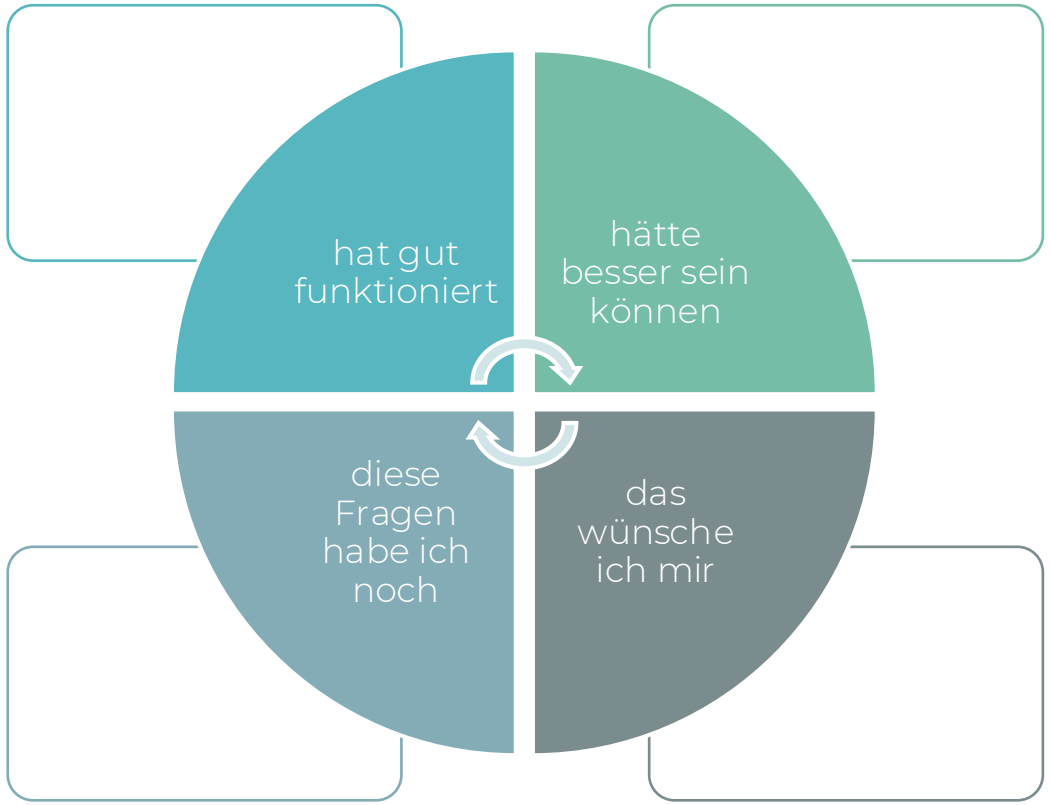
FEEDBACK TAG 4

WIE GEHT ES DIR?

Hier hast du die Möglichkeit, das heutige Seminar zu bewerten und mir so zu helfen, den nächsten Tag oder das nächste Seminar besser zu machen

zu langsam — Tempo — zu schnell

zu viel Theorie
zu wenig Praxis — Theorie vs Praxis — zu viel Praxis
zu wenig Theorie




07 FREITAG

RECAP TAG 4

RÜCKBLICK AUF DONNERSTAG

Das haben wir gemacht

...

Das solltest du jetzt können

...

Code zum Merken

...

Frage zum Aufwärmen

...

RECAP DER WOCHE

WAS WIR DIESE WOCHE GELERNT HABEN

Das haben wir gemacht

...

Das solltest du jetzt können

...

Code zum Merken

...

Frage zum Aufwärmen

...

ABSCHLUSS

WIE GEHT ES WEITER?

Weiterlesen

Begleitbuch von Michael Inden: die übrigen Kapitel

Offizielles Tutorial: https://docs.python.org/de/3/tutorial/

Weiterüben

Übungen mit den Lösungen in loesungen/ vergleichen

Tic-Tac-Toe erweitern: Computergegner, Punktestand, Spielfeldgröße

Übungsaufgaben mit Mentoring: https://exercism.org/tracks/python

Im Alltag

Eine kleine, lästige Aufgabe aus dem Job automatisieren: das beste Übungsprojekt

FEEDBACK ZUR WOCHE

WIE WAR DIE WOCHE?

Hier hast du die Möglichkeit, das heutige Seminar zu bewerten und mir so zu helfen, den nächsten Tag oder das nächste Seminar besser zu machen

zu langsam — Tempo — zu schnell

zu viel Theorie
zu wenig Praxis — Theorie vs Praxis — zu viel Praxis
zu wenig Theorie

